

Sawacom Ltd Cybersecurity Platform Code.

E-Portfolio link: https://github.com/Ojnymbok/Ojnyambok_Portfolio

Architectural Analysis, System Reflection & Design Mapping

In this capstone project, I set out to build a robust cybersecurity platform for Sawacom Ltd that addresses real-world mobile network vulnerabilities, particularly around SIM swap fraud, transaction security, and remote line protection. Drawing on object-oriented principles and established design patterns, I developed a Python-based simulation that enforces strict identity verification, cool-down periods, velocity checks, and USSD-driven whitelisting. The following sections reflect on the architectural decisions I made, the artefacts I produced, the challenges I encountered, and the trade-offs I considered during development.

1. Architectural & Strategy Highlights

How Design Patterns Supported Scalable Architecture

In designing the Sawacom platform, I chose the Observer Pattern to decouple core transaction processing and SIM lifecycle events from downstream modules like notifications and auditing (Gamma et al. 1994). In high-throughput telecom systems, tying events directly to business logic can create tight coupling and performance bottlenecks. By making `SawacomMNOEngine` the subject that publishes events to a registry of observers (via

`IRegistrationObserver`), I achieved clean separation. This design paid off in two key ways: it supports horizontal scaling—allowing new security handlers or AI microservices to be added at runtime without touching the core engine—and it makes asynchronous integration (e.g., with message brokers like Kafka) much more straightforward in a real deployment.

How Testing Strategies Ensured Robustness and Reliability

The validation strategy utilized a State-Driven and Boundary-Value Verification Pipeline targeting the four critical MNO security use cases viz SIM swap verification with IPRS (government ID database), cool-down periods, transaction velocity checks, and USSD-based remote locking.

I designed the test suite around state-driven and boundary-value testing to cover the four critical use cases. One practical challenge was validating the 72-hour cool-down window, since I couldn't wait days during testing. I solved this by programmatically shifting timestamps in the `SubscriberProfile` model. This approach let me confidently verify state transitions without leaks. I also deliberately injected malicious parameters (mismatched names and invalid document types) to confirm that regex validation and the `MockIPRSDatabase` rules catch issues early.

How AI-Readiness Was Considered at the Design Level

From the start, I wanted the architecture to be AI-ready so future intelligent models could plug in seamlessly (Kang 2019). I achieved this through abstraction barriers and interfaces like `NationalIdentityService`. The core engine only interacts with the abstract interface, so swapping the deterministic mock for a real biometric or ML-based verifier requires no changes to the main logic (Martin 2018). I also exposed calculated properties like `get_average_transaction()` on the `SubscriberProfile`, which makes feeding historical data as feature vectors into anomaly detection models much easier (Jordaan and von Solms 2011, Kang 2019).

2. Artefact Descriptions & Engineering Reflections

Artefact A: Core Domain Entities (`SubscriberProfile`)

Artefact A is the `SubscriberProfile` class, which encapsulates all sensitive state for a mobile line. I focused on strict encapsulation—marking fields like `_is_remote_locked` as private and controlling changes through properties—to enforce domain rules such as preventing future timestamps or negative transactions. One real-world challenge I anticipated was velocity attacks involving rapid micro-transactions (Lee et al. 2020). To keep things efficient, I limited the average calculation in `get_average_transaction()` to the last 10 records. This rolling window prevents memory issues while still catching anomalies effectively.

Artefact B: Core System Engine (`SawacomMNOEngine`)

In the central `SawacomMNOEngine`, I applied the Dependency Inversion Principle by injecting the `NationalIdentityService` through the constructor rather than hard-coding dependencies (Martin 2018). I also kept the engine focused on a single responsibility: enforcing compliance rules while delegating alerts to observers. Handling race conditions—such as a USSD lock happening at the same time as a branch SIM swap attempt—was tricky. I addressed it by placing an immutable check for `is_remote_locked` right at the start of `execute_sim_swap()`, which immediately blocks and notifies observers if needed.

Artefact C: Identity & Alerting Infrastructure (`MockIPRSDatabase` & `SecurityAlertNotificationEngine`)

I built infrastructure simulation components designed to fulfill government database verification queries and dispatch multi-channel security notifications whenever anomalous access events are detected.

Object-Oriented Principles & Techniques I Used

- Polymorphism: I implemented the `NationalIdentityService` interface via the `MockIPRSDatabase`. This allowed me to transparently substitute the mock implementation for a live government connection without disrupting the system.

- **Interface Segregation:** I had the alert engine implement the `IRegistrationObserver` interface. By doing this, I successfully kept the notification delivery mechanics completely separate from the core telecom state switching logic.

Challenges I Faced & My Resolutions

In actual network deployments, calling external government APIs introduces systemic latency and intermittent connection timeouts. I addressed this at the architectural level by isolating external boundaries behind an abstract interface. This enabled me to inject fallback loops, caching layers, or circuit breakers seamlessly, without altering the central business engine.

3. Explanatory Engineering Trade-offs

Pattern Selection Trade-offs (Observer Pattern)

The Observer pattern gave me clean decoupling and easy extensibility for future AI modules, which was a big win (Gamma et al. 1994). However, I learned that it introduces some opacity—tracing implicit event flows through the observer registry can be harder during debugging. I also had to be careful about wrapping observer calls in try-except blocks to prevent one failing handler from disrupting the main transaction pipeline.

Testing Complexity

Testing temporal behaviours like the exact 72-hour expiry was one of the more interesting challenges. Instead of stalling execution, I used time-shifting inside the test harness. While effective for verifying business logic, it required deeper access to internal properties, which adds some maintenance overhead. Overall, though, it gave me confidence in the system's reliability.

Managing Abstraction vs. Over-Engineering

Striking the right balance between abstraction and simplicity was key. Decoupling via `NationalIdentityService` was essential for future government or ML integrations (Jordaan and von Solms 2011). On the other hand, I deliberately avoided extra layers for simple models like `SubscriberProfile`—no Abstract Factories here. Adding unnecessary patterns would have created boilerplate without real benefits, so I kept those areas lean."

Conclusion

Overall, this experience reinforced for me the value of clean, extensible architecture and thoughtful testing in building secure systems. By prioritising abstraction, encapsulation, and behavioural patterns like Observer, I created a foundation that is not only functional today but ready for future enhancements such as real government API integration or machine learning-based fraud detection. This project strengthened my appreciation for balancing theoretical best practices with practical engineering constraints, and it has equipped me with valuable

insights I can carry into professional software development in the telecommunications security domain.

References

Gamma, E., Helm, R., Johnson, R. and Vlissides, J., 1994. *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley.

Jordaan, L. and von Solms, B., 2011. A Biometrics-Based Solution to Combat SIM Swap Fraud. *Lecture Notes in Computer Science*, vol. 6740, pp. 70-87.

Kang, H., 2019. *Fraud Detection in Mobile Money Transactions Using Machine Learning*. Master's thesis, Iowa State University.

Martin, R.C., 2018. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Boston, MA: Prentice Hall.

Lee, K., Kaiser, B., Mayer, J. and Narayanan, A., 2020. An Empirical Study of Wireless Carrier Authentication for SIM Swaps. In: Sixteenth Symposium on Usable Privacy and Security (SOUPS 2020). USENIX Association.